

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)
Кафедра экономической математики, информатики и статистики (ЭМИС)

БАЗОВЫЙ СИНТАКСИС КЛАССОВ

Отчёт по лабораторной работе по дисциплине «Объектно-ориентированное
программирование»

Студент гр.594-2

 А. А. Танцеров

“ 16 ” Октября 2025 г.

Руководитель канд. техн. наук, доцент
каф. ЭМИС ТУСУР

_____ Я. В. Костелей

“ _____ ” _____ 2025 г.

Томск 2025

Лабораторная работа №2

«Базовый синтаксис классов»

Цель работы: изучение принципов объектно-ориентированного программирования на языке C++ через разработку класса для управления данными.

Класс – это пользовательский тип данных, состоящий из методов и полей, схож с конструкцией структура (struct), но имеет свои особенности. Поле класса (свойство, атрибут) – переменная или переменная-объект, объявленная внутри класса. Метод – функция, объявленная внутри класса и используемая для манипуляции над значениями полей и реализации логики класса (функционала). Класс содержит поля и методы. Примером класса может выступать окно графического интерфейса. Что схожее между окнами? У них есть размеры, заголовки, они имеют логику открытия, закрытия, сворачивания, масштабирования и так далее. Можно предположить, что размеры и заголовок окна могут быть полями некоторого обобщенного (абстрактного) представления окна, а вот функции открытия, закрытия и изменения размера окна – методами класса, с помощью которых можно задать выполнение какой-то фиксированной логики на эти события (например, при изменении размеров окна хотим перегруппировать элементы управления внутри него). Мы понимаем, что есть что-то общее у всех окон, и что для реализации приложения нам может понадобиться создать несколько окон условно по одному шаблону. Поэтому можем сформировать один класс «Окно», а потом создавать экземпляры этого класса – объекты. Объект – это конкретный экземпляр заданного класса, или если сказать проще – это переменная сложного составного типа – класса. Объектов класса может быть много – а класс один. Другим примером класса может быть описание сотрудника. У сотрудников есть ряд общих важных для нас параметров: фамилия, имя, отчество, должность, размер заработной платы, паспортные данные, адрес, номер счета, его доступность, необходимая к выдаче зарплата

и другое, а также мы можем проводить с ним различные операции: начислять зарплату, отправлять в отпуск или командировку. Поэтому, представив эту сущность – «Сотрудник» – можно задать механизмы структурирования данных о сотруднике и их обработки в виде класса. Отчасти класс может перекликаться в описании предметной области с сущностью (таблицей) в базах данных, где класс похож на таблицу: его поля – на колонки-атрибуты таблицы, а строки – на экземпляры класса. Рассмотрим, как будет выглядеть класс для более простой сущности. Например, нам необходимо вести учет товара на складе. У товара есть название и его количество. Нам необходимо знать, сколько товара есть на данный момент на складе, а какой закончился. Определим класс «Product». В C++ класс обозначается ключевым словом `class`, далее следует имя, после чего внутри блоковых скобок определяются поля и методы. Объявление класса, как и структуры, должно заканчиваться точкой с запятой.

Задание 1. Сформируйте класс для хранения информации о приеме конкретного лекарства. Данный класс может содержать следующие поля и методы: - наименование лекарства (массив – `char*`); - день приема (номер дня с начала года); - количество приемов в день (равно 5); - план приема в заданный час суток (массив из 5 элементов) – отмечает необходимость приема лекарства в заданное время суток; - отметка приема (массив из 5 элементов) – отмечает прием лекарства в заданное время суток. Определить `enum` для времени суток из 5 элементов.

Конструктор с параметрами – наименование лекарства и день. Селекторы на все поля (кроме массивов). Метод вывода на консоль информации о приеме лекарства (включая план и фактический прием). Методы (отдельные) установки плана приема в определенное время суток и установки приема в определенное время суток, а также проверки необходимости принять лекарство в заданное время суток.

Метод проверки того, что фактический прием соответствует плану (лекарство в этот день приняли вовремя). Деструктор на освобождение

массивов (динамических). Определите, как минимум 6 исключений по вводу данных: некорректная срока, недопустимое время суток и т.д. Используйте ссылки, константные методы и модификатор `const` в нужных местах.

На первом этапе был создан класс `Medicine` для хранения информации о приёме лекарства в соответствии с техническим заданием. Определён перечислимый тип `TOD (TimeOfDay)` с пятью значениями: утро, перед обедом, обед, ужин, ночь. В классе объявлены приватные поля: динамическая строка `char* name` для названия лекарства, целое беззнаковое `unsigned int day` для номера дня с начала года, статическая константа `dosesPerDay = 5`, а также два динамических массива типа `bool* planReception` и `bool* didIt` для хранения плана и отметок о фактическом приёме. Для корректной работы с динамической памятью добавлен вспомогательный флаг `needToCopy`.

Были реализованы конструкторы с параметрами (включая конструктор копирования), деструктор для освобождения динамической памяти, селекторы `getName()` и `getDay()` (константные методы, возвращающие значения полей). Добавлены методы: `print()` для вывода всей информации на консоль, `setPlan()` и `setDid()` для установки плана и отметки о приёме в определённое время суток, `needToTake()` для проверки необходимости приёма, `checkPlan()` для проверки плана в заданное время и `isCorrect()` для проверки соответствия фактического приёма плану за весь день. Все методы, не изменяющие состояние объекта, помечены как `const`. Для обработки некорректных данных определены исключения: пустая строка названия, недопустимый день (меньше 1 или больше 365), неверный индекс времени суток (выход за границы массива), ошибки выделения памяти, попытка установки значения для неинициализированного массива и конфликт при копировании объектов.

В методах используются константные ссылки для параметров, где это необходимо для эффективности и безопасности кода. Код заголовочного файла представлен на листинге 1. Код реализации классов представлен на листинге 2.

```

#pragma once
class Medicine
{
private:
    char* name;
    unsigned int day;
    static const int requiredReception = 5;
    bool planReception[requiredReception] = {0};
    bool didIt[requiredReception] = {0};
    bool needToCopy;

    static const char* const timeNames[];
public:
    Medicine(const char* name, int day);
    Medicine(const char* name, int day, bool planReception, bool copy);
    ~Medicine();
    const char* const getName() const;
    int getDay() const;

    enum TOD {
        morning,
        before_dinner,
        dinner,
        supper,
        night
    };

    void print() const;
    void setPlan(TOD time, bool shouldTake);
    void setDid(TOD time, bool taken);
    bool needToTake(TOD time) const;
    bool checkPlan(TOD time) const;
    bool isCorrect(TOD time) const;
};

```

Листинг 1 – Код заголовочного файла класса Medicine

```

#include "medicine.h"
#include <string>
#include <iostream>
using namespace std;

const char* const Medicine::timeNames[] = { "óòõî", "äî îáääà", "îáää",
"óæëî", "îî÷ü" };

Medicine::Medicine(const char* medicalName, int startDay) { // TODO n è d

    // TODO n - nullptr, strlen(n), -> òðääîääîëÿ ê èîâèè (!)
    if (startDay < 1 || startDay > 365) {
        throw range_error("Âûðîäèò çà äèâîäçîí (îò 1 äî 365
âêëþ÷èòü!);");
    }
    if (medicalName == nullptr
        || strlen(medicalName) == 0
        || isdigit(medicalName[0])
        || ispunct(medicalName[0])) {
        throw invalid_argument("Îääåðîîå çîä÷äîèå îäçâîèëÿ èâèèðîòà!");
    }
    size_t medicines_name = strlen(medicalName);
    name = new char[medicines_name + 1];
    strcpy_s(name, medicines_name + 1, medicalName);
    day = startDay;
    for (int i = 0; i < requiredReception; i++) {

```

```

        didIt[i] = false;
        planReception[i] = false;
    }
}
Medicine::Medicine(const char* medicalName, int startDay, bool
planOfReceptions, bool copy) {
    if (!copy) {
        throw invalid_argument("Îøèáêà â êîìèðíââîèè!");
    }
    day = startDay++;
    size_t medicines_name = strlen(medicalName);
    name = new char[medicines_name + 1];
    strcpy_s(name, medicines_name + 1, medicalName);
}

Medicine::~Medicine() {
    delete[] name;
}

const char* const Medicine::getName() const {
    return name;
}

int Medicine::getDay() const {
    return day;
}

void Medicine::print() const{
    cout << "Íàèíâíââîèâ èâèàðñòâà: " << name << endl;
    cout << "Îðèýë â " << day << " äâü." << endl;
    for (int i = 0; i < requiredReception; i++) {
        cout << timeNames[i] << ": íââí - " << (planReception[i] ? "Äà;\t"
: "Íàð;\t");
        cout << "ñââèäë - " << (didIt[i] ? "Äà;\t" : "Íàð;\t") << endl;
    }
}

bool Medicine::checkPlan(TOD time) const{
    int index = static_cast<int>(time); //change enum to integer
    if (index < 0 || index >= requiredReception) {
        throw out_of_range("Ëíââñ âüðíäèò çà äðâîèöü îðè,ìà!");
    }
    if ((!planReception[index] && didIt[index] ) || (planReception[index]
&& !didIt[index] )) {
        return false;
    }
    return true;
}

void Medicine::setPlan(TOD time, bool shouldTake) {
    int index = static_cast<int>(time);
    if (index < 0 || index >= requiredReception) {
        throw out_of_range("Íââððíâ äðâý îðèâì!");
    }
    planReception[index] = shouldTake;
}

void Medicine::setDid(TOD time, bool taken) {
    int index = static_cast<int>(time);
    if (index < 0 || index >= requiredReception) {
        throw out_of_range("Íââððíâ äðâý îðèâì!");
    }
    didIt[index] = taken;
}

bool Medicine::needToTake(TOD time) const {
    int index = static_cast<int>(time);

```

```

        if (index < 0 || index >= requiredReception) {
            throw out_of_range("Íàäääðíîá âðáìÿ ïðåâîà!");
        }

        return planReception[index];
    }
    bool Medicine::isCorrect(TOD time) const{
        int index = static_cast<int>(time);
        if (index < 0 || index >= requiredReception) {
            throw out_of_range("Íàäääðíîá âðáìÿ ïðåâîà!");
        }

        return planReception[index];
    }
}

```

Листинг 2 – Код реализации методов класса Medicine

Вывод: изучены принципы объектно-ориентированного программирования на языке C++ через разработку класса для управления данными.